FULL-STACK ENGINEERING — WEEK 1

JavaScript for Developers

Variables, types, functions, scope, closures, ES6+, DOM manipulation, and async patterns — in one session.

WHAT WE'LL COVER

Agenda

01 Variables & Operators
const / let / var, strict equality

02 Data Types
Primitives, strings, arrays, objects

03 Functions
Declarations, arrows, HOFs, recursion

04 Scope & Closures
Block scope, hoisting, private state

05 ES6+ Features
Destructuring, spread, optional chaining

06 DOM Manipulation
Selecting, events, delegation

07 Async JavaScript
Event loop, promises, async/await

Three ways to declare

Keyword	Scope	Reassign?	Hoisted?
var	Function	Yes	undefined
let	Block	Yes	TDZ
const	Block	No	TDZ

> Use const by default. Use let when you need to reassign. Avoid var.

```
// Declaration
const name = "Alice";
let age = 25;
age = 26; // OK
```

strict-equality.js

```
// Strict vs Loose Equality
5 == "5" // true — converts types
5 === "5" // false — no conversion
```

block-scope.js

```
if (true) {
 var leaked = "escapes!";
 let contained = "stays here";
}
leaked; // "escapes!"
contained; // ReferenceError
```

Seven primitives

`string` `number` `boolean` `undefined` `null` `bigint` `symbol`

Falsy values

Everything else is truthy — these six are the only falsy values:

`false` `0` `""` `null` `undefined` `NaN`

quirks.js

```
// JS quirks you should know
typeof null // "object" — famous bug
typeof NaN // "number"
0.1 + 0.2 // 0.30000000000004
```

coercion.js

```
// Type coercion surprises
"5" + 3 // "53" — concatenation
"5" - 3 // 2 — math
true + true // 2
```

template-literals.js

```
const name = "Alice";
const msg = `Hello, ${name}!`;
```

The iteration methods

array-methods.js

```
const arr = [1, 2, 3, 4, 5];
arr.map(n => n * 2) // [2, 4, 6, 8, 10]
arr.filter(n => n > 2) // [3, 4, 5]
arr.reduce((s, n) => s + n, 0) // 15
arr.find(n => n > 3) // 4
```

`.map — transforms` `.filter — keeps matches` `.reduce — single value` `.find — first match`

objects.js

```
const person = {
 name: "Alice",
 age: 25
};
person.name // "Alice"
person["age"] // 25
```

object-methods.js

```
Object.keys(person) // ["name", "age"]
Object.values(person) // ["Alice", 25]
// Spread — copy & extend
const updated = { ...person, age: 26 };
```

Three syntaxes

Declaration

Hoisted

```
function add(a, b) {
 return a + b;
}
```

Expression

Not hoisted

```
const sub = function(a, b) {
 return a - b;
};
```

Arrow (ES6)

No own this

```
const mul = (a, b) ⇒ a * b;
// Implicit return
const dbl = n ⇒ n * 2;
```

Default & Rest params

```
function greet(name, g = "Hello") {
 return `${g}, ${name}!`;
}
function sum(...nums) {
 return nums.reduce((s,n) ⇒ s+n, 0);
}
```

Factory pattern (HOF)

```
function createMultiplier(factor) {
 return (n) ⇒ n * factor;
}
const triple = createMultiplier(3);
triple(5); // 15
```

Functions remember

A closure is a function that remembers the variables from the scope where it was created, even after that scope has finished executing.

- Global — accessible everywhere
- Function — inside the function only
- Block — inside { } with let/const

> TDZ: let/const are hoisted but not initialized — accessing them before declaration throws a ReferenceError.

Closure in action

```
function createCounter() {
 let count = 0; // private!
 return {
 increment: () => ++count,
 getCount: () => count
 };
}
```

Memoize pattern

```
function memoize(fn) {
 const cache = {};
 return (...args) => {
 const key = JSON.stringify(args);
 if (key in cache) return cache[key];
 return cache[key] = fn(...args);
 };
}
```

Destructuring & modern syntax

Object destructuring

```
const { name, age } = person;
// Rename + defaults
const { name: fullName,
 country = "Unknown" } = person;
```

Array destructuring

```
const [first, ...rest] = [1,2,3];
// Swap without temp
[a, b] = [b, a];
```

Spread & Rest

```
// Spread — expand
[...arr1, 4, 5]
{ ...obj1, c: 3 }
// Rest — collect
function f(...args)
```

Optional chaining ?.

```
// Short-circuits on null/undefined
user?.address?.city
user?.getProfile?.()
arr?.[0]
```

Nullish coalescing ??

```
// Defaults only for null/undefined
0 ?? "default" // 0
0 || "default" // "default"
"" ?? "default" // ""
"" || "default" // "default"
```

Select, modify, listen

Selecting

```
document.querySelector("#hero")
document.querySelectorAll(".link")
```

Modifying

```
el.textContent = "New text";
el.dataset.tab // reads data-tab="..."
```

classList API

```
el.classList.add("active")
el.classList.toggle("open")
```

Event delegation

```
// One listener on parent catches all
document.querySelector("ul")
 .addEventListener("click", (e) => {
 if (e.target.tagName === "LI")
 e.target.classList.toggle("done");
 });
```

Create & remove

```
// Create → configure → attach
const li = document.createElement("li");
li.textContent = "New item";
list.appendChild(li);
li.remove(); // gone
```

Forms

```
form.addEventListener("submit", (e) => {
 e.preventDefault(); // stop reload
 form.reset();
});
```

The Event Loop

JS is single-threaded — one call stack, one thing at a time. The event loop coordinates async work between the JS engine and the browser/Node runtime.

1 **Call Stack** Runs your code, one frame at a time

↓ empty?

2 **Microtask Queue** Promise .then(), drain ALL first

↓ empty?

3 **Task Queue** setTimeout, I/O — pick ONE, repeat

Predict the output

```
console.log("1");
setTimeout(() =>
 console.log("2"), 0);
Promise.resolve().then(() =>
 console.log("3"));
console.log("4");
```

Output: 1 4 3 2

1, 4 sync (call stack) → 3 microtask (Promise) → 2 macrotask (setTimeout)

From callbacks to async/await

Callback

Nests deep

```
getUser(1, (err, user) => {
 getPosts(user.id, (err, p) => {
 getComments(p[0].id,
 (err, c) => { // pyramid
 });
 });
});
```

Promise chain

Flat

```
getUser(1)
 .then(u => getPosts(u.id))
 .then(p => getComments(p[0].id))
 .then(c => console.log(c))
 .catch(e => console.error(e));
```

async/await

Reads like sync

```
async function main() {
 const u = await getUser(1);
 const p = await getPosts(u.id);
 const c = await getComments(p[0].id);
}
```

Error handling + Parallel

```
async function safeFetch(url) {
 try { return { ok: true, data: await fetch(url) };
 } catch (e) { return { ok: false, error: e.message } }
}
// Parallel — start all, await together
const [a, b] = await Promise.all([
 fetch(url1), fetch(url2)
]);
```

Promise static methods

Method	Resolves
`.all`	All fulfill (fail fast)
`.allSettled`	All settle (never rejects)
`.race`	First to settle
`.any`	First to fulfill